```
import pandas as pd                          # Data manipulation
import re                                    # Regex for text cleaning
import pickle                                # Serialising model & vectorizer
import time                                  # Timing the pipeline
from sklearn.model_selection import train_test_split   # Train/test split
from sklearn.feature_extraction.text import TfidfVectorizer  # TF-IDF
from sklearn.linear_model import LogisticRegression     # Classifier
from sklearn.naive_bayes import MultinomialNB            # Alternate classifier
from sklearn.metrics import accuracy_score, classification_report  # Eval
from sklearn.pipeline import Pipeline                    # (bonus: pipeline)
```

```
# ----------------------------------------------------------------------------
# 1.  LOAD THE DATASET
# ----------------------------------------------------------------------------
# Adjust the path if the file lives in a different location in Colab
# (e.g., "/content/drive/MyDrive/dolly_15k.csv" for Google Drive)
DATASET_PATH = "dolly_15k.csv"

print("=" * 70)
print("STEP 1 — Loading the dataset")
print("=" * 70)

df = pd.read_csv(DATASET_PATH)
```

```
======================================================================
STEP 1 — Loading the dataset
======================================================================
```

```
# ----------------------------------------------------------------------------
# 2.  BASIC EXPLORATORY DATA ANALYSIS (EDA)
# ----------------------------------------------------------------------------
print("\n" + "=" * 70)
print("STEP 2 — Dataset overview")
print("=" * 70)

# 2a. Shape — number of rows and columns
print(f"\nDataset shape : {df.shape[0]} rows × {df.shape[1]} columns")

# 2b. Column names (helps identify the correct text & label columns)
print(f"\nColumn names  : {list(df.columns)}")

# 2c. First 5 rows so we can visually inspect the data
print("\n--- First 5 rows ---")
print(df.head())
```

```
======================================================================
STEP 2 — Dataset overview
======================================================================

Dataset shape : 15011 rows × 4 columns

Column names  : ['instruction', 'context', 'response', 'category']

--- First 5 rows ---
                                     instruction  \
0         When did Virgin Australia start operating?
1           Which is a species of fish? Tope or Rope
2     Why can camels survive for long without water?
3  Alice's parents have three daughters: Amy, Jes...
4                    When was Tomoaki Komorida born?

                                         context  \
0  Virgin Australia, the trading name of Virgin A...
1                                            NaN
2                                            NaN
3                                            NaN
4  Komorida was born in Kumamoto Prefecture on Ju...

                                      response        category
0  Virgin Australia commenced services on 31 Augu...       closed_qa
1                                         Tope  classification
2  Camels use the fat in their humps to keep them...         open_qa
3            The name of the third daughter is Alice         open_qa
4         Tomoaki Komorida was born on July 10,1981.       closed_qa
```

```
# 2d. Missing values per column
print("\n--- Missing values ---")
```

```
print(df.isnull().sum())
```

```
--- Missing values ---
instruction        0
context        10545
response           0
category           0
dtype: int64
```

```
df = df.drop(columns=['context'])
```

```
# 2d. Missing values per column
print("\n--- Missing values ---")
print(df.isnull().sum())
```

```
--- Missing values ---
instruction    0
response       0
category       0
dtype: int64
```

```
# 2e. Distribution of categories / labels
#     (we assume the label column is called "category" — dolly_15k standard)
LABEL_COL = "category"
TEXT_COL   = "instruction"
```

```
if LABEL_COL in df.columns:
    print(f"\n--- Category distribution ---")
    print(df[LABEL_COL].value_counts())
else:
    # Fallback: try to find a column that looks like labels
    possible_label_cols = [c for c in df.columns if c.lower() in ("category", "label", "label_name", "output")]
    if possible_label_cols:
        LABEL_COL = possible_label_cols[0]
        print(f"\n[INFO] Auto-detected label column: '{LABEL_COL}'")
        print(df[LABEL_COL].value_counts())
    else:
        raise KeyError(
            "Could not find a label column. "
            "Please set LABEL_COL manually to match your CSV."
        )
```

```
--- Category distribution ---
category
open_qa                   3742
general_qa                2191
classification            2136
closed_qa                 1773
brainstorming             1766
information_extraction    1506
summarization             1188
creative_writing           709
Name: count, dtype: int64
```

```
# Confirm text column exists
if TEXT_COL not in df.columns:
    possible_text_cols = [c for c in df.columns if "instruction" in c.lower() or "prompt" in c.lower() or "text" in c.lower
    if possible_text_cols:
        TEXT_COL = possible_text_cols[0]
        print(f"[INFO] Auto-detected text column: '{TEXT_COL}'")
    else:
        raise KeyError(
            "Could not find a text/instruction column. "
            "Please set TEXT_COL manually to match your CSV."
        )
```

```
print(f"\nUsing  →  Text column : '{TEXT_COL}'")
print(f"Using  →  Label column: '{LABEL_COL}'")
```

```
Using  →  Text column : 'instruction'
Using  →  Label column: 'category'
```

```
# Drop rows where either text or label is missing (safety net)
df = df.dropna(subset=[TEXT_COL, LABEL_COL]).reset_index(drop=True)
print(f"Rows after dropping NaNs: {len(df)}")
```

```
Rows after dropping NaNs: 15011
```

```
# ----------------------------------------------------------------------------
# 3.  TEXT CLEANING
#     - Convert to lowercase
#     - Remove special characters & numbers
#     - Collapse extra whitespace
# ----------------------------------------------------------------------------
print("\n" + "=" * 70)
print("STEP 3 — Cleaning text data")
print("=" * 70)
```

```
======================================================================
STEP 3 — Cleaning text data
======================================================================
```

```
def clean_text(text: str) -> str:
    """
    Clean a single string:
      1. Lowercase
      2. Remove every character that is NOT a letter or a space  (removes
         numbers, punctuation, special symbols)
      3. Collapse multiple spaces into a single space
      4. Strip leading/trailing whitespace
    """
    # Ensure we are working with a string (handles NaN that slipped through)
    text = str(text)

    # Step 1: lowercase
    text = text.lower()

    # Step 2: keep only a-z and spaces
    text = re.sub(r"[^a-z\s]", " ", text)

    # Step 3: collapse multiple spaces → single space
    text = re.sub(r"\s+", " ", text)

    # Step 4: strip
    text = text.strip()

    return text
```

```
# Apply the cleaning function to the entire text column
df["clean_text"] = df[TEXT_COL].apply(clean_text)

# Quick sanity check — show a before/after example
print("\n--- Before / After cleaning (first 3 samples) ---")
for i in range(3):
    print(f"\nOriginal : {df[TEXT_COL].iloc[i][:120]}...")
    print(f"Cleaned  : {df['clean_text'].iloc[i][:120]}...")
```

```
--- Before / After cleaning (first 3 samples) ---

Original : When did Virgin Australia start operating?...
Cleaned  : when did virgin australia start operating...

Original : Which is a species of fish? Tope or Rope...
Cleaned  : which is a species of fish tope or rope...

Original : Why can camels survive for long without water?...
Cleaned  : why can camels survive for long without water...
```

```
# ----------------------------------------------------------------------------
# 4.  DEFINE FEATURES (X) AND LABELS (y)
# ----------------------------------------------------------------------------
print("\n" + "=" * 70)
print("STEP 4 — Defining features (X) and labels (y)")
print("=" * 70)

X = df["clean_text"]       # Feature: cleaned instruction text
y = df[LABEL_COL]          # Label  : category string

print(f"X shape: {X.shape}")
```

```
print(f"y shape: {y.shape}")
print(f"Unique labels ({len(y.unique())}): {sorted(y.unique())}")
```

```
======================================================================
STEP 4 — Defining features (X) and labels (y)
======================================================================
X shape: (15011,)
y shape: (15011,)
Unique labels (8): ['brainstorming', 'classification', 'closed_qa', 'creative_writing', 'general_qa', 'information_extractio
```

```
# ---------------------------------------------------------------------------
# 5.  TRAIN / TEST SPLIT  (stratified to preserve class balance)
# ---------------------------------------------------------------------------
print("\n" + "=" * 70)
print("STEP 5 — Splitting into train / test sets")
print("=" * 70)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,           # 20 % held out for testing
    random_state=42,          # Reproducibility
    stratify=y                # Keep label ratios identical in both splits
)

print(f"Training set  : {len(X_train)} samples")
print(f"Testing set   : {len(X_test)} samples")
```

```
======================================================================
STEP 5 — Splitting into train / test sets
======================================================================
Training set  : 12008 samples
Testing set   : 3003 samples
```

```
# ---------------------------------------------------------------------------
# 6.  TF-IDF VECTORIZATION
#     Convert raw text → numerical feature matrix
# ---------------------------------------------------------------------------
print("\n" + "=" * 70)
print("STEP 6 — TF-IDF Vectorization")
print("=" * 70)

# We use unigrams + bigrams, cap features at 50 000 for memory efficiency,
# and ignore extremely rare terms (min_df=2 → appears in ≥ 2 documents).
vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),      # unigrams + bigrams
    max_features=50000,      # limit vocabulary size
    min_df=2,                # ignore very rare words
    max_df=0.95,             # ignore very common words (near-stopwords)
    sublinear_tf=True        # apply log scaling to term frequency
)

# Fit on training data ONLY, then transform both train and test
start = time.time()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf  = vectorizer.transform(X_test)
elapsed = time.time() - start

print(f"Vocabulary size : {len(vectorizer.vocabulary_)}")
print(f"Training matrix : {X_train_tfidf.shape}")
print(f"Testing matrix  : {X_test_tfidf.shape}")
print(f"Vectorization time: {elapsed:.2f} seconds")
```

```
======================================================================
STEP 6 — TF-IDF Vectorization
======================================================================
Vocabulary size : 19300
Training matrix : (12008, 19300)
Testing matrix  : (3003, 19300)
Vectorization time: 0.66 seconds
```

```
# ---------------------------------------------------------------------------
# 7.  TRAIN THE CLASSIFIER  (Logistic Regression)
#     Alternative: MultinomialNB — uncomment the NB block to switch.
# ---------------------------------------------------------------------------
print("\n" + "=" * 70)
```

```
print("STEP 7 — Training the classifier")
print("=" * 70)

# --- Option A: Logistic Regression (recommended for this dataset) ---
model = LogisticRegression(
    solver="lbfgs",
    max_iter=500,
    n_jobs=-1,              # use all CPU cores
    random_state=42
)

# --- Option B: Multinomial Naive Bayes (uncomment to use instead) ---
# model = MultinomialNB(alpha=0.1)

start = time.time()
model.fit(X_train_tfidf, y_train)
elapsed = time.time() - start

print(f"Model trained in {elapsed:.2f} seconds")
print(f"Model type: {model.__class__.__name__}")
```

```
======================================================================
STEP 7 — Training the classifier
======================================================================
Model trained in 7.77 seconds
Model type: LogisticRegression
```

```
# ------------------------------------------------------------------------------
# 8.  EVALUATION — Accuracy & Full Classification Report
# ------------------------------------------------------------------------------
print("\n" + "=" * 70)
print("STEP 8 — Model evaluation")
print("=" * 70)

# Predict on the held-out test set
y_pred = model.predict(X_test_tfidf)

# 8a. Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"\nOverall Accuracy: {acc:.4f}  ({acc*100:.2f} %)\n")

# 8b. Detailed classification report (precision, recall, f1 per class)
print("--- Classification Report ---")
print(classification_report(y_test, y_pred, digits=4))
```

```
======================================================================
STEP 8 — Model evaluation
======================================================================

Overall Accuracy: 0.6670  (66.70 %)

--- Classification Report ---
                        precision    recall  f1-score   support

         brainstorming     0.7447    0.7025    0.7230       353
        classification     0.9435    0.9391    0.9413       427
             closed_qa     0.7500    0.5577    0.6397       355
      creative_writing     0.9000    0.5070    0.6486       142
            general_qa     0.5012    0.4726    0.4865       438
information_extraction     0.8507    0.5681    0.6813       301
               open_qa     0.5223    0.7984    0.6315       749
         summarization     0.7606    0.4538    0.5684       238

              accuracy                         0.6670      3003
             macro avg     0.7466    0.6249    0.6650      3003
          weighted avg     0.7018    0.6670    0.6669      3003
```

```
# ------------------------------------------------------------------------------
# 9.  SAVE THE TRAINED MODEL  →  model.pkl
# ------------------------------------------------------------------------------
print("=" * 70)
print("STEP 9 — Saving the trained model")
print("=" * 70)

MODEL_PATH = "model.pkl"
with open(MODEL_PATH, "wb") as f:
    pickle.dump(model, f)

print(f"Model saved to '{MODEL_PATH}'")
```

```
======================================================================
STEP 9 — Saving the trained model
======================================================================
Model saved to 'model.pkl'
```

```
# ------------------------------------------------------------------------
# 10. SAVE THE TF-IDF VECTORIZER  →  vectorizer.pkl
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("STEP 10 — Saving the TF-IDF vectorizer")
print("=" * 70)

VECTORIZER_PATH = "vectorizer.pkl"
with open(VECTORIZER_PATH, "wb") as f:
    pickle.dump(vectorizer, f)

print(f"Vectorizer saved to '{VECTORIZER_PATH}'")
```

```
======================================================================
STEP 10 — Saving the TF-IDF vectorizer
======================================================================
Vectorizer saved to 'vectorizer.pkl'
```

```
# ------------------------------------------------------------------------
# BONUS: Quick function to demonstrate loading & predicting on new text
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("BONUS — Demo: load saved artifacts & predict on new text")
print("=" * 70)


def predict_category(new_text: str, vectorizer_path: str, model_path: str) -> str:
    """
    Load the saved vectorizer and model, clean the input text,
    vectorize it, and return the predicted category.
    """
    # Load artifacts
    with open(vectorizer_path, "rb") as f:
        vec = pickle.load(f)
    with open(model_path, "rb") as f:
        clf = pickle.load(f)

    # Clean & transform
    cleaned = clean_text(new_text)
    features = vec.transform([cleaned])

    # Predict
    prediction = clf.predict(features)[0]

    # Also show top-3 probabilities (Logistic Regression supports this)
    if hasattr(clf, "predict_proba"):
        probs = clf.predict_proba(features)[0]
        top_idx = probs.argsort()[::-1][:3]
        print("Top-3 predictions:")
        for idx in top_idx:
            label = clf.classes_[idx]
            prob  = probs[idx]
            print(f"  {label:25s}  {prob:.4f}")

    return prediction


# Try a couple of example prompts
examples = [
    "Write a short poem about the ocean at sunset.",
    "What is the capital of France?",
    "Summarize the key points of the US Constitution.",
    "Can you brainstorm some ideas for a birthday party?"
]

for ex in examples:
    print(f"\nInput : {ex}")
    pred = predict_category(ex, VECTORIZER_PATH, MODEL_PATH)
    print(f"Predicted category: {pred}")
```

```
======================================================================
BONUS — Demo: load saved artifacts & predict on new text
======================================================================
```

```
Input : Write a short poem about the ocean at sunset.
Top-3 predictions:
  creative_writing           0.7852
  general_qa                 0.0661
  summarization              0.0354
Predicted category: creative_writing

Input : What is the capital of France?
Top-3 predictions:
  open_qa                    0.4638
  general_qa                 0.1907
  closed_qa                  0.1126
Predicted category: open_qa

Input : Summarize the key points of the US Constitution.
Top-3 predictions:
  summarization              0.8829
  open_qa                    0.0392
  general_qa                 0.0228
Predicted category: summarization

Input : Can you brainstorm some ideas for a birthday party?
Top-3 predictions:
  brainstorming              0.7473
  general_qa                 0.1225
  open_qa                    0.0446
Predicted category: brainstorming
```

```
# ------------------------------------------------------------------------
# END OF PIPELINE
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("Pipeline complete!")
print("=" * 70)
```

```
======================================================================
Pipeline complete!
======================================================================
```

Start coding or generate with AI.